CBCC

Codex-Bound Constitutional Cryptography

Security Architecture Specification

> **Core doctrine: Proof Before Plaintext. No Root to Steal.**

Version: Architecture Baseline 1.0

Status: Reference security architecture for implementation, review, threat modeling, and integration.

Scope: Defines the security model, trust boundaries, cryptographic governance flow, root-defense architecture, proof requirements, authority structure, failure behavior, and validation requirements for CBCC. It intentionally does not prescribe any specific host platform or application integration.

CBCC treats plaintext creation as a governed cryptographic event. Possession of ciphertext, key material, policy state, a codex, or an individual authority contribution is insufficient by itself to cause protected information to become intelligible.

1. Architecture Objective

CBCC is a cryptographic-governance architecture designed to make authorization, authority, semantic context, policy state, and execution integrity cryptographically consequential. Its objective is not to replace established cryptographic primitives. Its objective is to change the conditions under which protected information may become plaintext.

> **Security objective: Prevent any single compromised component, operator, process, authority, host, or secret store from independently creating a valid decryption event.**

The architecture therefore rejects the conventional assumption that a protected master key can safely exist somewhere as a complete retrievable object. CBCC instead models root authority as a distributed cryptographic function whose effect can only be exercised when a valid set of independently verifiable conditions converge.

2. Security Principles

2.1 Proof Before Plaintext

A decryption request does not begin with key retrieval. It begins with proof construction and verification. Plaintext may only be instantiated after all required cryptographic, constitutional, authority, semantic, temporal, and execution proofs validate.

2.2 The Key Is Insufficient

A conventional content key, recipient secret, codex copy, or authority credential must not be sufficient by itself. Every sensitive derivation must be bound to a specific artifact, operation, recipient or requester, purpose, authority state, policy state, codex state, execution measurement, epoch, nonce, and expiration.

2.3 No Reconstructable Root

The root of trust must never exist as a complete value in process memory, storage, backups, logs, environment variables, configuration, or administrative tooling. Root authority is distributed across independent trust domains and exercised only through threshold computation.

2.4 Fail Closed

Ambiguity, missing proof, stale state, signature failure, attestation failure, contradiction, replay, policy mismatch, codex mismatch, or audit discontinuity produces denial. Unknown is not equivalent to authorized.

2.5 Independent Verification

Trusted decisions are accepted as proofs, not claims. A field such as LEGITIMATE, VERIFIED, ACTIVE, or UNBYPASSED has no security meaning unless the receiving verifier can independently validate the evidence that produced it.

3. Threat Model

CBCC assumes that attackers may obtain substantial partial access. The architecture is specifically intended to remain resistant when one or more ordinary system layers have already failed.

- Attacker may possess ciphertext and complete public artifact metadata.
- Attacker may possess the codex and understand its schema and traversal rules.
- Attacker may compromise an application process or low-privilege identity.
- Attacker may steal one authority credential or one threshold contribution.
- Attacker may replay previously observed requests, approvals, or protocol messages.
- Attacker may tamper with policy, codex contents, runtime configuration, or audit records.
- Attacker may compromise one administrative domain or one CI/CD path.
- Attacker may gain code execution in a non-root orchestration process.

The design must continue to deny plaintext unless the attacker can satisfy the actual distributed authorization conditions required by the architecture.

4. Trust Boundaries

CBCC should be decomposed into distinct trust domains. Collapsing these domains into one process or administrative boundary defeats the root-defense model.

Domain	Primary responsibility	Must not possess
Orchestrator	Construct requests, collect proofs, route operations	Root shares, unrestricted content keys
Policy verifier	Evaluate canonical policy inputs	Root shares, plaintext
Codex verifier	Verify codex root, edition, Merkle proofs, traversal	Root shares, unrestricted decryption authority
Authority domains	Independently verify proof package and produce threshold contribution	Other authority shares, complete root
Sealed worker	Perform one approved operation on protected content	Exportable root, reusable unrestricted authority
Audit authority	Commit signed evidence of request, decision, and result	Plaintext or raw secrets

5. Root-Defense Architecture

> **Root-defense invariant: No component may possess enough information to reconstruct the root, derive unrestricted descendant keys, and access plaintext.**

CBCC replaces root reconstruction with distributed threshold computation. Each authority holds a non-exportable share or hardware-bound secret contribution. The authorities jointly evaluate a cryptographic function over a canonical authorization transcript. The root itself is never reconstructed.

```
INCORRECT:
shares -> reconstruct root -> derive content key

CBCC MODEL:
distributed authority shares -> threshold evaluation -> operation-bound result
-> root never reconstructed
```

Suitable constructions include threshold pseudorandom functions, distributed key generation, threshold KEMs, threshold signatures used as authorization inputs, and threshold decryption protocols. The exact primitive must be selected from established, reviewed cryptographic work rather than invented ad hoc.

5.1 Authority Separation

Root authorities must be independent in both technical and administrative terms. A credible deployment avoids shared failure domains across host, cloud account, operating environment, deployment pipeline, secrets manager, logging plane, administrator set, and signing infrastructure.

5.2 Non-Exportable Shares

Authority shares should be generated and retained inside HSMs, TPM-backed services, secure enclaves, dedicated cryptographic appliances, or equivalently constrained hardware roots. APIs must not expose raw share export, arbitrary signing, arbitrary key derivation, or arbitrary decryption.

5.3 Structured Operations Only

An authority should accept only canonical, typed requests such as VERIFY_CONSTITUTIONAL_PROOF, APPROVE_OPERATION, PRODUCE_THRESHOLD_CONTRIBUTION, REVOKE_EPOCH, or REFRESH_SHARE. The authority must reject opaque arbitrary byte operations that bypass semantic and domain separation.

6. Constitutional Derivation Request

Every protected operation is represented by a canonical proof-bearing request. The request is not itself trusted; it is the object independently verified by each participating authority.

```
ConstitutionalDerivationRequest {
format_version
artifact_id
ciphertext_commitment
requester_identity_proof
capability_grant
purpose_proof
codex_root
policy_root
traversal_proof
interpretation_proof
execution_measurement
authority_manifest
epoch
revocation_state
requested_operation
nonce
issued_at
expires_at
}
```

Every field that influences authorization must be cryptographically bound into the resulting threshold transcript or operation key. Any field omitted from binding becomes a substitution or replay surface.

7. Codex-Bound Governance

The codex is a canonical, versioned, signed corpus whose structure participates directly in authorization and derivation. It is not merely documentation. Each relevant clause is addressable, canonicalized, hashed, and committed into a Merkle structure.

7.1 Codex Properties

- Stable clause identifiers and deterministic canonical byte encoding.
- Edition identity and edition lifecycle state.
- Per-clause commitments and Merkle inclusion proofs.
- Authority signatures over edition roots or volume roots.
- Reference integrity for semantic edges and traversal dependencies.
- Revocation, succession, and declassification state.

7.2 Governed Semantic Traversal

Authorization may require traversal through codex relationships rather than static coordinates. A traversal must resolve deterministically and produce a portable proof of every visited clause and semantic edge used in the decision.

```
artifact classification
-> required authority class
-> applicable purpose clause
-> temporal restriction
-> active exception
-> recipient continuity requirement
-> final admissibility state
```

Every visited node should include an independently verifiable inclusion proof against the bound codex root. Ambiguous or missing edges fail closed.

8. Policy and Purpose Proofs

Policy admissibility and purpose legitimacy are separate decisions and should remain separately provable. Policy answers whether the requested operation is structurally admissible. Purpose reflection establishes whether the declared reason for the operation satisfies the applicable constraints.

Purpose must not be a caller-controlled string that directly satisfies authorization. It should resolve to a signed or otherwise independently verifiable proof object whose claims are bound to the artifact, operation, requester, policy root, codex root, epoch, and expiry.

9. Interpretation Proof

Semantic interpretation must not be represented as opaque caller-supplied bytes. The architecture should require a canonical, signed, independently verifiable object.

```
InterpretationProof {
format_version
codex_root
traversal_commitment
policy_decision_id
purpose_decision_id
semantic_input_commitment
resolved_meaning_commitment
resolver_identity
resolver_version
determinism_profile
issued_at
expires_at
authority_scope
signature
}
```

The verifier must validate the schema, canonical encoding, codex binding, traversal binding, policy binding, resolver authorization, determinism profile, freshness, and signature before the interpretation proof can affect derivation.

10. Threshold Authority

Authority quorum is not simply a count of servers. High-assurance deployments should require heterogeneous authority classes so that multiple copies of the same technical role cannot manufacture legitimacy.

- Constitutional or policy authority.
- Security or containment authority.
- Execution-integrity authority.
- Human, institutional, or recovery authority where required.

Each threshold contribution must be bound to the exact request transcript. A contribution valid for one artifact, purpose, operation, worker measurement, or epoch must be useless in another.

11. Operation-Bound Derivation

The cryptographic result produced by the distributed authorities should be specific to one approved operation. The derivation context should include, at minimum: artifact identifier, ciphertext commitment, requester or recipient, purpose decision, requested operation, codex root, policy root, execution measurement, epoch, nonce, expiration, and current audit-chain head.

```
operation_key = ThresholdFunction(
distributed_root,
H(artifact_id || ciphertext_commitment || requester || purpose ||
operation || codex_root || policy_root || worker_measurement ||
epoch || nonce || expires_at || audit_head)
)
```

The operation key should not be a reusable descendant of a long-lived master key visible to application code. It should exist only inside the approved execution boundary and only for the lifetime of the authorized operation.

12. Attested Execution Boundary

A valid governance transcript must not release cryptographic capability to arbitrary code. The threshold result should be delivered only to an approved execution environment whose identity can be independently measured and attested.

- Immutable or measured boot state.
- Approved worker binary and dependency set.
- Verified policy and codex verifier versions.
- Restricted debugging and process attachment.
- Restricted filesystem and network access.
- Output-channel enforcement.
- Short-lived session binding and freshness checks.

The trusted worker should perform the approved operation and release only the permitted result. Unrestricted plaintext export should not be the default API.

```
Preferred interface:
execute_governed_operation(artifact, operation, proof) -> constrained_result

Avoid:
decrypt(artifact) -> unrestricted_plaintext
```

13. Replay Resistance and Freshness

Every request and threshold contribution must be non-replayable. Fresh nonces, expiration, epoch binding, consumed-request registration, and audit-head binding should prevent reuse of prior approvals.

A captured approval for one operation must not authorize another. For example, an approval to verify a claim must not be convertible into permission to export the full protected artifact.

14. Epochs, Rotation, and Proactive Refresh

Long-lived static threshold material creates unacceptable compromise accumulation. Root authority should operate in explicit epochs, with proactive share refresh or equivalent distributed key update procedures that do not reconstruct the root.

- Old authority shares become cryptographically useless after refresh.
- Compromises separated across epochs cannot be combined trivially.
- Artifact and operation derivation remain bound to the active epoch.
- Revocation state is signed and independently verifiable.

15. Audit Architecture

Every success and failure must extend a signed, hash-linked audit chain. Audit evidence should establish what was requested, which proofs were presented, which authorities participated, which decision was reached, and what output commitment was produced.

Audit records must never contain plaintext, raw root shares, raw content keys, or other reconstructive secret material.

15.1 Minimum Audit Fields

- Request commitment and canonical request identifier.
- Identity, capability, purpose, policy, codex, and traversal proof commitments.
- Authority decisions and threshold transcript commitments.
- Execution measurement and attestation result.
- Final decision and operation scope.
- Output commitment, not unrestricted output content.
- Key-destruction or session-termination attestation where applicable.

16. Fail-Closed Conditions

The following conditions require immediate denial before plaintext creation or continuation of a protected operation:

- Unknown, revoked, stale, or mismatched codex edition or root.
- Missing or invalid Merkle proof for any traversal dependency.
- Unresolvable or ambiguous semantic traversal.
- Invalid or stale policy root.
- Missing, malformed, expired, or unauthorized purpose proof.
- Insufficient or invalid authority quorum.
- Replay or nonce reuse.
- Execution measurement mismatch.
- Attestation failure or post-attestation state change.
- Audit-chain discontinuity.
- Revocation-state mismatch.
- Any cryptographic signature or commitment verification failure.

17. Compromise Containment Expectations

CBCC is designed so that compromise of an ordinary application layer does not automatically expose protected information. The following are expected outcomes under the intended architecture.

Compromise	Expected outcome
Ciphertext stolen	No plaintext without valid governed derivation.
Codex stolen	Structure becomes known; authority remains unavailable.
One authority compromised	Single contribution insufficient; authority quarantined and epoch refreshed.
Application process compromised	Cannot reconstruct root or obtain operation key without valid proof and approved worker attestation.
Old approval replayed	Rejected by nonce, epoch, expiry, and transcript binding.
Policy file modified	Policy-root mismatch; derivation denied.
Codex clause modified	Codex-root or inclusion-proof mismatch; derivation denied.
Worker modified	Execution measurement mismatch; no threshold release.

18. Cryptographic Primitive Guidance

CBCC should compose established primitives and place its novelty in governance binding, proof structure, distributed authority, derivation context, and execution control. Candidate primitives include authenticated encryption such as XChaCha20-Poly1305 or AES-GCM-SIV; HKDF-SHA-512 for domain-separated expansion; Argon2id where memory-hard derivation is appropriate; Ed25519 or standardized post-quantum signature schemes for signatures; X25519, standardized KEMs, or hybrid KEM constructions for recipient encapsulation; Merkle trees for codex commitments; and mature threshold-cryptography protocols for distributed authority.

Security-critical finite-field, threshold, signature, KEM, and secret-sharing implementations should be independently reviewed and validated against external implementations. Custom cryptographic primitives should be avoided.

19. Security Invariants

An implementation claiming conformance to this architecture should maintain the following invariants:

- No reconstructable root exists in ordinary application memory or storage.
- No single authority can authorize or derive a protected operation alone.
- Every authorization-relevant field is bound into the cryptographic transcript.
- Every trusted state is independently verifiable evidence, not a caller assertion.
- Every protected operation is limited in artifact, purpose, scope, environment, epoch, and time.
- No stale or replayed authority contribution can be reused successfully.
- Plaintext is created only inside an approved execution boundary.
- Unrestricted plaintext export is never the implicit default.
- Every success and failure produces tamper-evident evidence.

- Ambiguity and contradiction fail closed.

20. Verification and Validation Requirements

Before production use, the architecture requires security validation beyond unit-test success.

- Formal protocol specification and explicit adversary model.
- Independent cryptographic review of threshold construction and transcript binding.
- External red-team assessment with application, host, insider, replay, supply-chain, and authority-compromise scenarios.
- Independent verifier implementation to detect shared implementation assumptions.
- Hardware-attestation validation and downgrade testing.
- Recovery, revocation, disaster-recovery, and availability analysis.
- Property-based and randomized testing of canonicalization, Merkle proofs, quorum handling, replay resistance, and state transitions.
- Fault-injection testing across authority failures and partial compromise.
- Audit verification against tamper, truncation, reordering, and signature substitution.

21. Reference Security Flow

```
REQUEST
-> IDENTITY PROOF
-> CAPABILITY PROOF
-> CODEX VERIFICATION
-> POLICY RESOLUTION
-> PURPOSE PROOF
-> SEMANTIC TRAVERSAL PROOF
-> INTERPRETATION PROOF
-> AUTHORITY QUORUM
-> DISTRIBUTED THRESHOLD EVALUATION
-> ATTESTED SEALED OPERATION
-> OUTPUT CONSTRAINT ENFORCEMENT
-> AUDIT ATTESTATION
-> SESSION / OPERATION KEY DESTRUCTION
```

22. Architectural Definition

> **CBCC definition: Codex-Bound Constitutional Cryptography is a proof-bearing cryptographic governance architecture in which plaintext admissibility depends on the verified convergence of distributed authority, constitutional policy, semantic codex state, purpose legitimacy, identity and capability proof, temporal state, and execution integrity. The root of trust is represented as a distributed cryptographic function rather than a reconstructable key.**

The defining security shift is from protecting a master secret to preventing any single compromised system from manufacturing a valid decryption reality.

23. Doctrine

> **Proof Before Action.**
> **Proof Before Plaintext.**
> **No Root to Steal.**